Week 1 · Class 1: Practice Exercises — ANSWER KEY

Introduction to Quantitative Political Analysis

2025-12-31

1 Segment A — Introduction to Quantitative Political Analysis

2 Non-AI Exercises

These are short reflection questions graded on effort. The model answers below capture the key ideas; students should be credited for engaging thoughtfully, not for matching this wording.

2.1 1. The Purpose of This Course

In your own words, what is *quantitative* political analysis, and why might it matter even for someone who never works in politics professionally?

Answer: **Quantitative political analysis is the systematic use of numbers, data, and statistical methods to understand political questions** — testing theories, measuring public opinion, evaluating whether policies actually work, and predicting behavior, rather than relying on intuition alone. The course's goal is to help you (1) think about politics scientifically, (2) work with data to answer political questions, and (3) communicate findings clearly.

It matters even outside a political career because it makes you a **more informed citizen**. You can critically evaluate the "data" claims politicians make, understand what polls really mean (and when to be skeptical), recognize misleading statistics in advertising, and make sense of election forecasts and their uncertainty.

2.2 2. Why Scientific Testing Is Important

Why is systematically testing an idea against data more reliable than trusting intuition or "conventional wisdom"? You may use an example (such as Dr. John Snow and the 1854 cholera outbreak) to illustrate your point.

Answer: **Conventional wisdom and intuition can be confidently wrong.** The scientific method guards against this by forcing us to observe patterns, form a hypothesis, **test it against evidence**, replicate, and revise. Data lets us check our beliefs instead of assuming them.

John Snow is the classic example. In 1854 the accepted theory was that cholera spread through "bad air" (miasma). Rather than argue from intuition, Snow **systematically mapped every cholera death** and the locations of public water pumps. The data revealed that deaths clustered around the Broad Street pump (578 deaths nearby vs. a handful elsewhere). That evidence overturned the dominant theory and led authorities to remove the pump handle, ending the outbreak. Without the data, the connection would have stayed hidden in a list of scattered addresses.

2.3 3. The Role of AI in This Class

In this course, AI handles much of the technical work of writing code. What does that free *you* to focus on, and why might those skills be more valuable than memorizing syntax?

Answer: **We use AI to handle technical implementation so you can focus on the intellectual work** that actually drives good analysis:

- **Understanding concepts** and how they apply to real political problems
- **Interpreting results** and their political implications
- **Asking the right questions** and designing a sensible research strategy
- **Recognizing the limitations** of both the data and the methods
- **Communicating findings** clearly to different audiences

These skills are more durable than syntax because tools change constantly — a skilled analyst doesn't spend time remembering whether it's `filter()` or `subset()`; they spend it deciding what's worth measuring and whether the answer makes sense. AI can produce code, but **judgment** about what to ask and whether to trust the result is the part you have to supply.

2.4 4. Responsible Use of AI

AI is powerful but not infallible. Describe **two** ways an AI could produce an answer that is technically correct but still misleading, and explain why you should always verify what the AI gives you.

Answer: AI can write code that **runs perfectly yet leads to a wrong conclusion.** Examples (any two):

- **Correlation vs. causation** — it may report a real correlation and imply one thing causes another when it doesn't.
- **Missing context** — it doesn't understand the political or social background and can misinterpret what a number means.
- **Biased data** — it can't tell when the underlying data is unrepresentative or skewed, so a "correct" calculation rests on a flawed sample.
- **Wrong method** — it may apply a statistical technique that's inappropriate for the question.
- **Hallucination** — it can confidently invent data, results, or sources that don't exist.

You verify because **the code being error-free is not the same as the analysis being right.** Your job is to develop the judgment to notice when a result seems off and to check it against what you know.

3 AI Exercises

> **Before you start:** Some of these tasks ask you to do things in R that you haven't learned yet — and that's on purpose. You are **not** expected to already know how to write this code. Your job is to work *with* Claude to get there, and to pay attention to what the code does. Don't worry if it feels unfamiliar — **we will cover all of this in depth in the coming weeks.** These exercises are graded on **effort, not correctness**, so experiment, make mistakes, and focus on learning.

Tips for Working with Claude:

- Ask for **R code using only tidyverse** (no other packages)
- Request **simple, focused answers** to your specific question—not complex analyses
- Ask Claude to **explain what the code is doing** since you're learning
- Avoid asking for visualizations or plots in these exercises
- Include the output of `glimpse()` in your prompt so Claude knows your variable names

Example prompt: "Using tidyverse in R, calculate the mean age. Keep the code simple and explain what each line does. Here is what my data looks like: [paste glimpse output]"

*Note: this is **simulated** data. Several results below are deliberately close to flat or run against real-world expectations — a useful reminder that you should always read the numbers carefully rather than assume the AI's output matches your priors.*

3.1 5. National Political Attitudes

Dataset: data/nat_pol_attitudes.csv

Description: Simulates a nationally representative survey measuring political attitudes, ideology, and demographics.

Variables:

- `respondent_id`: Unique respondent ID (int)
- `age`: Age in years, 18-90 (int)
- `gender`: male, female, nonbinary (factor)
- `race_ethnicity`: White, Black, Latino, Asian, Other (factor)
- `education`: Less than HS, HS, Some College, BA, Postgrad (ordered)
- `income_bracket`: Ten brackets from <$10k to >$200k (ordered)
- `ideology`: 1 (very liberal) to 7 (very conservative) (int)
- `party_id`: Democrat, Republican, Independent, Other (factor)
- `trust_gov`: 0-10 political trust scale (int)
- `policy_support_env`: Support for environmental regulation, 0/1 (binary)
- `policy_support_guns`: Support for stricter gun laws, 0/1 (binary)

3.1.1 5.1 Load the Data

We'll start at the very beginning: getting the data into R and taking a first look at it.

```
# Load the dataset
nat_pol_attitudes <- read_csv("data/nat_pol_attitudes.csv")
```

```
Rows: 1200 Columns: 11
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (3): gender, race_ethnicity, party_id
dbl (8): respondent_id, age, education, income_bracket, ideology, trust_gov,...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Explore the structure of the data
glimpse(nat_pol_attitudes)
```

```
Rows: 1,200
Columns: 11
$ respondent_id       <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,~
$ age                 <dbl> 36, 34, 32, 36, 44, 41, 81, 40, 60, 73, 69, 23, 87~
$ gender              <chr> "male", "female", "female", "female", "female", "f~
$ race_ethnicity      <chr> "White", "White", "Latino", "Other", "White", "Whi~
$ education           <dbl> 4, 5, 2, 2, 4, 1, 5, 3, 1, 3, 4, 2, 4, 1, 4, 4, 5,~
$ income_bracket      <dbl> 2, 10, 5, 10, 2, 1, 7, 4, 4, 7, 8, 7, 9, 10, 4, 6,~
$ ideology            <dbl> 5, 5, 3, 2, 6, 6, 5, 4, 4, 1, 3, 2, 4, 4, 6, 1, 3,~
$ party_id            <chr> "Republican", "Independent", "Independent", "Repub~
$ trust_gov           <dbl> 2, 2, 5, 0, 5, 6, 5, 4, 4, 4, 2, 4, 9, 3, 3, 4, 0,~
$ policy_support_env  <dbl> 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1,~
$ policy_support_guns <dbl> 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0,~
```

3.1.2 5.2 Average Age

Ask Claude to help you calculate the **average (mean) age** of all respondents in the dataset.

```
# Calculate the overall mean age
nat_pol_attitudes %>%
  summarise(avg_age = mean(age, na.rm = TRUE))
```

```
# A tibble: 1 x 1
  avg_age
    <dbl>
1    53.2
```

The average respondent is about 53 years old (53.2).

3.1.3 5.3 Average Age by Party

Now build on that. Ask Claude to calculate the **average age separately for each party** (`party_id`). How do the parties compare?

```
# Group by party, then take the mean age within each group
nat_pol_attitudes %>%
  group_by(party_id) %>%
  summarise(
    n = n(),
    avg_age = mean(age, na.rm = TRUE)
  ) %>%
  arrange(desc(avg_age))
```

```
# A tibble: 4 x 3
  party_id        n avg_age
  <chr>       <int>   <dbl>
1 Other          36    54.7
2 Republican    314    54.2
3 Independent   393    53.4
4 Democrat      457    52.2
```

The parties are close in age: Democrats average about 52.2, Independents 53.4, Republicans 54.2, and the small "Other" group 54.7. The spread is only about two years, so age differences across parties are modest in this sample.

3.1.4 5.4 Average Trust in Government by Party

Using the same idea, ask Claude to calculate the **average `trust_gov` score for each party**. Which party trusts government the most?

```r
nat_pol_attitudes %>%
  group_by(party_id) %>%
  summarise(
    avg_trust = mean(trust_gov, na.rm = TRUE)
  ) %>%
  arrange(desc(avg_trust))
```

```
# A tibble: 4 x 2
  party_id    avg_trust
  <chr>           <dbl>
1 Other            4.31
2 Democrat         4.15
3 Independent      4.08
4 Republican       3.79
```

Trust in government is fairly similar across parties (all roughly 3.8–4.3 on the 0–10 scale). Democrats (4.15) and Independents (4.08) trust slightly more than Republicans (3.79). The differences are small.

3.1.5 5.5 Support for Environmental Regulation

`policy_support_env` is 0/1 (1 = supports the policy). The **average of a 0/1 variable is a rate** — the share of people who support it. Ask Claude to calculate the **overall share of respondents who support environmental regulation**.

```r
# The mean of a 0/1 column is the proportion of 1s
nat_pol_attitudes %>%
  summarise(env_support_rate = mean(policy_support_env, na.rm = TRUE))
```

```
# A tibble: 1 x 1
  env_support_rate
             <dbl>
1            0.717
```

About 72% of respondents (0.72) support environmental regulation.

3.1.6 5.6 Gun-Law Support by Party

Ask Claude to calculate the **share who support stricter gun laws** (`policy_support_guns`) **for each party**. (Look closely at the result — does the pattern match what you expected? Keep this in mind for the next dataset.)

```
nat_pol_attitudes %>%
  group_by(party_id) %>%
  summarise(
    gun_support_rate = mean(policy_support_guns, na.rm = TRUE)
  ) %>%
  arrange(desc(gun_support_rate))
```

```
# A tibble: 4 x 2
  party_id    gun_support_rate
  <chr>                  <dbl>
1 Republican             0.341
2 Independent            0.331
3 Democrat               0.304
4 Other                  0.25
```

Support for stricter gun laws is low and similar across parties (roughly 25–34%). Notably, in this simulated data Republicans show the highest rate (0.34), slightly above Democrats (0.30) — the opposite of the real-world pattern. This is a good example of why you should not assume the AI's output matches your expectations: the code is correct, but the data is synthetic, so the substantive story does not reflect reality.

3.2 6. Election Returns by Precinct

Dataset: data/precinct_elections.csv

Description: Precinct-level election returns with demographics.

Variables:

- `state`: Two-letter state abbreviation (factor)
- `county`: County name (string)
- `precinct_id`: Unique precinct identifier (int)
- `year`: Election year (int)
- `reg_voters`: Number of registered voters (int)
- `turnout`: Number of voters who turned out (count) (int)

- dem_votes: Democratic candidate votes (int)
- rep_votes: Republican candidate votes (int)
- median_income: Precinct median household income (num)
- pct_bachelor: Percentage with bachelor's degree (num)
- race_black: Percentage Black population (num)
- race_hispanic: Percentage Hispanic population (num)

3.2.1 6.1 Load the Data

```r
# Load the dataset
precinct_elections <- read_csv("data/precinct_elections.csv")
```

```
Rows: 3000 Columns: 12
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (3): state, county, precinct_id
dbl (9): year, reg_voters, turnout, dem_votes, rep_votes, median_income, pct...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Examine the data
glimpse(precinct_elections)
```

```
Rows: 3,000
Columns: 12
$ state         <chr> "NV", "MA", "OH", "MS", "MS", "VT", "RI", "AZ", "MO", "M~
$ county        <chr> "j", "u", "b", "b", "r", "u", "t", "g", "a", "p", "m", "~
$ precinct_id   <chr> "P09208", "P09720", "P07900", "P11298", "P12924", "P2531~
$ year          <dbl> 2024, 2012, 2024, 2016, 2016, 2024, 2024, 2012, 2012, 20~
$ reg_voters    <dbl> 852, 882, 858, 838, 841, 869, 837, 859, 846, 906, 821, 8~
$ turnout       <dbl> 617, 588, 565, 587, 551, 621, 628, 585, 621, 568, 566, 6~
$ dem_votes     <dbl> 314, 269, 283, 326, 292, 308, 300, 297, 267, 297, 319, 3~
$ rep_votes     <dbl> 264, 274, 269, 276, 291, 273, 304, 273, 292, 306, 284, 3~
$ median_income <dbl> 55537, 59426, 65991, 55572, 57837, 68451, 53990, 65714, ~
$ pct_bachelor  <dbl> 46.4, 44.5, 27.0, 36.2, 20.9, 32.0, 11.7, 39.1, 15.0, 65~
$ race_black    <dbl> 48.2, 55.3, 69.7, 5.7, 38.3, 57.6, 66.7, 61.2, 49.7, 15.~
$ race_hispanic <dbl> 73.0, 58.8, 8.7, 23.9, 71.5, 9.7, 44.3, 57.7, 58.4, 25.6~
```

3.2.2 6.2 Average Turnout Rate

The `turnout` column is a **count** of voters, not a percentage. To get a turnout *rate*, divide it by the number of registered voters: `turnout / reg_voters`. Ask Claude to calculate the **average turnout rate across all precincts**.

```
# Build the rate first, then average it across precincts
precinct_elections %>%
  mutate(turnout_rate = turnout / reg_voters) %>%
  summarise(avg_turnout_rate = mean(turnout_rate, na.rm = TRUE))
```

```
# A tibble: 1 x 1
  avg_turnout_rate
             <dbl>
1            0.707
```

Average turnout is about 71% of registered voters (0.71). *(Note: don't compute turnout from* ***`dem_votes + rep_votes`*** *— those columns are generated independently and can exceed* ***`turnout`****. Use the* ***`turnout`*** *column directly.)*

3.2.3 6.3 Turnout Rate by Income

Do richer precincts vote at higher rates? Ask Claude to split precincts into **four groups by `median_income`** (income quartiles) and calculate the **average turnout rate within each group**. What do you find?

```
# ntile() splits median_income into 4 equal-sized groups (quartiles)
precinct_elections %>%
  mutate(
    turnout_rate = turnout / reg_voters,
    income_quartile = ntile(median_income, 4)
  ) %>%
  group_by(income_quartile) %>%
  summarise(
    n_precincts = n(),
    avg_turnout_rate = mean(turnout_rate, na.rm = TRUE)
  )
```

```
# A tibble: 4 x 3
  income_quartile n_precincts avg_turnout_rate
```

```
           <int>     <int>         <dbl>
1              1       750         0.706
2              2       750         0.706
3              3       750         0.707
4              4       750         0.708
```

Turnout is essentially flat across income quartiles (all 0.706–0.708). In this dataset there is no meaningful relationship between precinct income and turnout — a reminder that "no difference" is itself a valid, and common, finding.

3.3 7. Congressional Press Releases

Dataset: data/congress_press.csv

Description: Corpus of press releases issued by U.S. legislators.

Variables:

- `release_id`: Unique press-release ID (int)
- `member_id`: Legislator ID (int)
- `chamber`: House, Senate (factor)
- `party`: Democrat, Republican, Independent (factor)
- `ideology_score`: DW-NOMINATE first dimension (num)
- `state`: Two-letter abbreviation (factor)
- `date`: Release date (date)
- `topic`: Ten topics like Health, Economy, Foreign Policy, etc. (factor)
- `sentiment_score`: -1 to 1 sentiment scale (num)
- `contains_attack`: Indicator of partisan attack language (binary)

3.3.1 7.1 Load the Data

```r
# Load the dataset
congress_press <- read_csv("data/congress_press.csv")
```

```
Rows: 2200 Columns: 10
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (4): chamber, party, state, topic
dbl  (5): release_id, member_id, ideology_score, sentiment_score, contains_a...
date (1): date
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Look at the data structure
glimpse(congress_press)
```

```
Rows: 2,200
Columns: 10
$ release_id      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,~
$ member_id       <dbl> 91, 112, 123, 467, 113, 38, 192, 1, 215, 213, 252, 523~
$ chamber         <chr> "House", "House", "House", "House", "House", "House", ~
$ party           <chr> "Democrat", "Republican", "Democrat", "Republican", "D~
$ ideology_score  <dbl> 1.00833230, 0.03292990, -0.29090319, -0.18102718, 0.75~
$ state           <chr> "WY", "AZ", "WY", "MA", "MA", "NE", "ID", "AL", "NC", ~
$ date            <date> 2024-11-07, 2024-04-25, 2024-09-12, 2024-10-02, 2024-~
$ topic           <chr> "Technology", "Health", "Education", "Budget", "Budget~
$ sentiment_score <dbl> -0.20, 0.12, 0.18, -0.16, 0.35, -0.44, -0.19, 0.60, 0.~
$ contains_attack <dbl> 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, ~
```

3.3.2 7.2 Attack-Language Rate by Party

`contains_attack` is 0/1, so its average is a rate. Ask Claude to calculate the **share of press releases that contain attack language, for each party**. Which party uses attack language most often in this data?

```r
congress_press %>%
  group_by(party) %>%
  summarise(
    attack_rate = mean(contains_attack, na.rm = TRUE)
  ) %>%
  arrange(desc(attack_rate))
```

```
# A tibble: 3 x 2
  party       attack_rate
  <chr>             <dbl>
1 Republican        0.579
2 Democrat          0.569
3 Independent       0.494
```

Roughly half of all press releases contain attack language, and the rate is nearly tied between the two major parties (Republicans 0.58, Democrats 0.57); the small Independent group is a bit lower (0.49). The Democrat–Republican gap is about one percentage point — too small to read much into.

3.3.3 7.3 Average Sentiment by Party

Finally, ask Claude to calculate the **average `sentiment_score` for each party**. Are the differences large or small?

```r
congress_press %>%
  group_by(party) %>%
  summarise(
    avg_sentiment = mean(sentiment_score, na.rm = TRUE)
  ) %>%
  arrange(desc(avg_sentiment))
```

```
# A tibble: 3 x 2
  party       avg_sentiment
  <chr>               <dbl>
1 Democrat          0.00736
2 Republican       -0.0116
3 Independent      -0.0216
```

All parties hover right around neutral (scores within about ± 0.02 of zero on the -1 to 1 scale): Democrats slightly positive ($+0.01$), Republicans and Independents slightly negative (-0.01 and -0.02). The differences are negligible.